

Neural Ordinary Differential Equations

Alek Dimitriev

University of Texas at Austin, McCombs School of Business

November 19, 2019

Table of Contents

1 Introduction

- Deep Learning Revolution
- Residual Networks
- Ordinary Differential Equations

2 Neural ODEs

- Motivation
- Training
- Applications

3 Augmented Neural ODEs

- Motivation
- Approach

Deep learning revolution

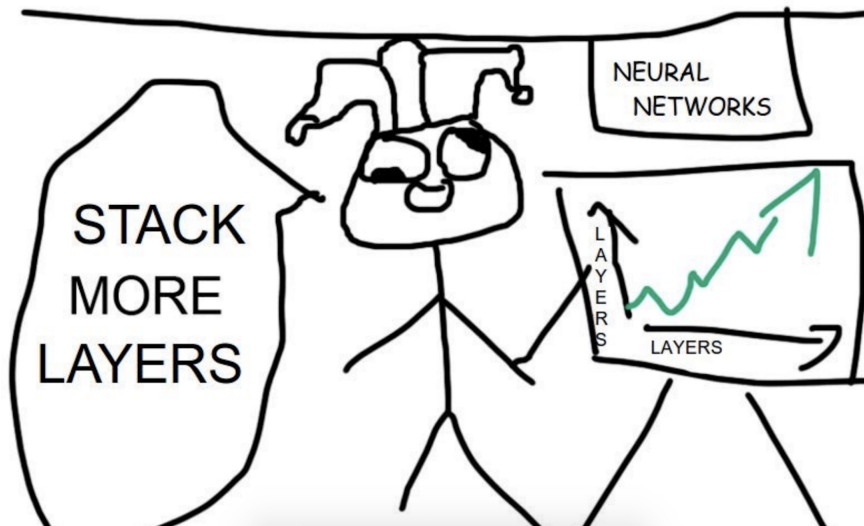


Figure: reddit.com/r/ProgrammerHumor/comments/8cli45/stack_more_layers/

Deep learning: a short history

- Neural networks used to be shallow for a reason
- Theoretical – universal approximation theorem¹
- Practical – vanishing gradient
- However, adding layers launched the deep learning revolution
- Empirically – pre-training and ReLU solved the gradient problem, followed by ResNets
- Theoretically – width- $(|\text{input}| + 4)$ ReLU networks are universal approximators²

¹Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural networks 2.5* (1989), pp. 359–366.

²Zhou Lu et al. “The expressive power of neural networks: A view from the width”. In: *Advances in neural information processing systems*. 2017, pp. 6231–6239.

Residual networks³

- ResNets add the input to the output at each layer/block:

$$x_{n+1} = F(x_n) + x_n$$

- If $H(x)$ is the target, it is easier to approximate the residual $H(x) - x$ by $F(x)$
- Identity mapping means $F(x) = 0$, easier than $F(x) = x$
- But why does this solve the gradient problem?

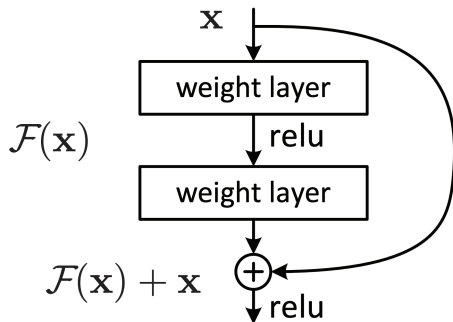


Figure: Fig. 2 in ResNets

³Kaiming He et al. "Deep residual learning for image recognition". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

Vanilla neural network gradients

- Let the input be $x = x_0$, with intermediate layer outputs $x_1 = F_1(x_0)$, $x_2 = F_2(x_1)$ and output $y = F_3(x_2)$ and $F_n(x) = w_n x$ for simplicity
- Vanilla net with two hidden layers:
 $y = F_3(x_2) = F_3(F_2(x_1)) = F_3(F_2(F_1(x_0)))$
- Taking the gradient:

$$\frac{dy}{dw_1} = \frac{dF_3(x_2)}{dx_2} \frac{dx_2}{dw_1} = \frac{dF_3(x_2)}{dx_2} \frac{dF_2(x_1)}{dx_1} \frac{dx_1}{dw_1} = w_3 w_2 x_0$$

Residual network gradients

- Now let $x_1 = F_1(x_0) + x_0$, $x_2 = F_2(x_1) + x_1$ and $y = F_3(x_2) + x_2$
- ResNet:

$$\begin{aligned}y &= F_3(x_2) + x_2 \\&= F_3(F_2(x_1) + x_1) + F_2(x_1) + x_1 \\&= F_3(F_2(F_1(x_0) + x_0) + F_1(x_0) + x_0) + F_2(F_1(x_0) + x_0) + F_1(x_0) + x_0\end{aligned}$$

- Taking the gradient:

$$\frac{dy}{dw_1} = w_3 w_2 x_0 + w_3 x_0 + w_2 x_0 + x_0$$

- Let $h(t)$ be a function and $h' = \frac{dh}{dt}$, i.e. $h^{(n)}(t) = \frac{d^n h}{dt^n}$.
- General definition:
$$f\left(t, h(t), h'(t), \dots, h^{(n-1)}(t)\right) = h^{(n)}(t)$$
- Simple first-order example:
$$f(t, h(t)) = 5h(t) - 3 \implies 5h - 3 = h'$$
- What is the solution? ($h \propto e^{5t}$)
- Can we solve it numerically?

Euler's method

- Let's solve a first-order ODE $h' = f(h, t)$ with a given initial condition $h(0) = c$
- Taylor approximation: $h(t + \alpha) = h(t) + \alpha h'(t) + \dots$
- Plugging in: $h(t + \alpha) \approx h(t) + \alpha f(t, h(t))$
- Euler's method: $h_{n+1} = h_n + \alpha f(h_n, t_n)$
- Example: $h'(t) = 5h(t) - 3$ with $h(0) = 1$ and $\alpha = 1$
- Table of solutions:

n	t	$h_n(t)$	$h'_n(t)$
0	0	1	2
1	1	3	12
2	2	15	72

Table of Contents

1 Introduction

- Deep Learning Revolution
- Residual Networks
- Ordinary Differential Equations

2 Neural ODEs

- Motivation
- Training
- Applications

3 Augmented Neural ODEs

- Motivation
- Approach

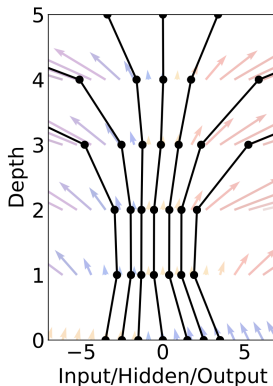
ResNets vs. Euler's method

ResNets	Euler's method
$x_{n+1} = x_n + F_{n+1}(x_n)$	$h_{n+1} = h_n + \alpha f(h_n, t_n)$

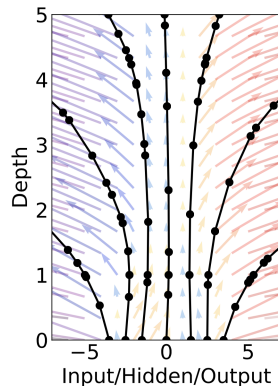
- Can we model "depth" $\rightarrow \infty$?
- Let the gradient be a neural network:

$$\frac{dz(t)}{dt} = f(z(t), t, \theta)$$

Residual Network



ODE Network



The adjoint method⁴

$$L(z(T)) = L\left(z(0) + \int_0^T f(z(t), t, \theta) dt\right) = L\left(\text{ODE}(z(0), f, 0, T, \theta)\right)$$

- Don't naively backpropagate through the ODE solver
- Solve a time-reversed ODE to get the gradient
- Define and solve a new ODE $a(t) = \partial L / \partial z(t)$:

$$\frac{da(t)}{dt} = -\frac{\partial f(z(t), t, \theta)}{\partial z} a(t), \quad a(T) = \frac{\partial L}{\partial z(T)}$$

- The gradient with respect θ and the input are then:

$$\frac{dL}{d\theta} = \int_0^T a(t)^T \frac{\partial f(z(t), t, \theta)}{\partial \theta} dt$$

⁴Tian Qi Chen et al. "Neural ordinary differential equations". In: *Advances in neural information processing systems*. 2018, pp. 6571–6583.

Proof of the adjoint sensitivity method

Let's rewrite L and define a new function $\mathcal{L}$:

$$L(z) = \int_0^T l(z, t, \theta) dt + L(z_0), \quad l(z, t, \theta) = \frac{dL(z_t)}{dt} = \frac{\partial L^T(z_t)}{\partial z} f(z_t, t, \theta)$$

$$\mathcal{L}(z, \frac{\partial z}{\partial t}, \theta) := \int_0^T l(z, t, \theta) dt + \int_0^T \lambda^T \left(\frac{\partial z}{\partial t} - f(z, t, \theta) \right) dt$$

$$\frac{d\mathcal{L}}{d\theta} = \int_0^T \nabla_z l^T \frac{\partial z}{\partial \theta} + \nabla_\theta l dt + \int_0^T \lambda^T \frac{\partial}{\partial \theta} \frac{\partial z}{\partial t} - \lambda^T \frac{\partial f}{\partial z} \frac{\partial z}{\partial \theta} - \lambda^T \frac{\partial f}{\partial \theta} dt$$

Per partes:
$$\int_0^T \lambda^T \frac{\partial z}{\partial \theta} dt = \lambda_T^T \frac{\partial z_T}{\partial \theta} - \lambda_0^T \frac{\partial z_0}{\partial \theta} - \int_0^T \frac{\partial \lambda^T}{\partial t} \frac{\partial z_t}{\partial \theta} dt$$

Proof of the adjoint sensitivity method

$$\frac{d\mathcal{L}}{d\theta} = \int_0^T \nabla_{\theta} l dt + \left(\nabla_z l - \frac{\partial \lambda^T}{\partial t} - \lambda^T \frac{\partial f}{\partial z} \right) \frac{\partial z}{\partial \theta} dt - \lambda^T \frac{\partial f}{\partial \theta} dt + \lambda_T^T \frac{\partial z_T}{\partial \theta}$$

Define a new differential equation:

$$\frac{\partial \lambda^T}{\partial t} = \nabla_z l - \lambda^T \frac{\partial f}{\partial z}, \quad \lambda_T^T = 0$$

The gradient then simplifies to:

$$\nabla_{\theta} \mathcal{L} = \int_0^T \left(\nabla_z L^T - \lambda^T \right) \frac{\partial f}{\partial \theta} dt$$

If we let $a(t) = \nabla_z L - \lambda(t)$, then we obtain the adjoint equation from before⁵.

⁵Proof adapted from docdroid.net/i4DgLv9/neural-odes.pdf

Algorithm

Putting it all together:

- ➊ Given: data (x, y) , NN dynamics $f_\theta(z(t), t)$, loss L
- ➋ Forward pass: with initial condition $z(0) = x$, integrate:

$$z(T) = z(0) + \int_0^T f_\theta(z(t), t) dt$$

- ➌ Backward pass: with initial condition $a(T) = \partial L(z(T))/\partial z$, integrate:

$$a(0) = a(T) + \int_T^0 a(t) \frac{\partial f_\theta(z(t), t)}{\partial z} dt$$

- ➍ Integrate:

$$\frac{\partial L}{\partial \theta} = \int_0^T a(t) \frac{\partial f_\theta(z(t), t)}{\partial \theta} dt$$

- ➎ Return: gradients w.r.t. input: $a(0) = \partial L(z(0))/\partial z$, params: $\partial L/\partial \theta$.

Neural ODE instead of ResNet blocks

	Test Error	# Params	Memory	Time
1-Layer MLP	1.60%	0.24M		
ResNet	0.41%	0.60M	$\mathcal{O}(L)$	$\mathcal{O}(L)$
RK-Net	0.47%	0.22M	$\mathcal{O}(\tilde{L})$	$\mathcal{O}(\tilde{L})$
ODE-Net	0.42%	0.22M	$\mathcal{O}(1)$	$\mathcal{O}(\tilde{L})$

- RK-net is backpropagating the gradient through a Runge-Kutta solver
- Adjoint method memory cost independent of "depth"

- Instead of # of layers, specify accuracy of ODE (error tolerance)
- Depth " $\approx$ " # of function evaluations
- Constant memory cost

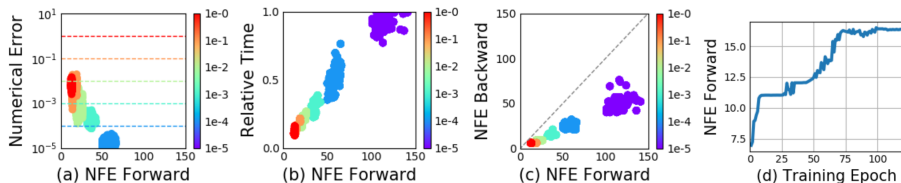


Figure 3: Statistics of a trained ODE-Net. (NFE = number of function evaluations.)

Irregular time series

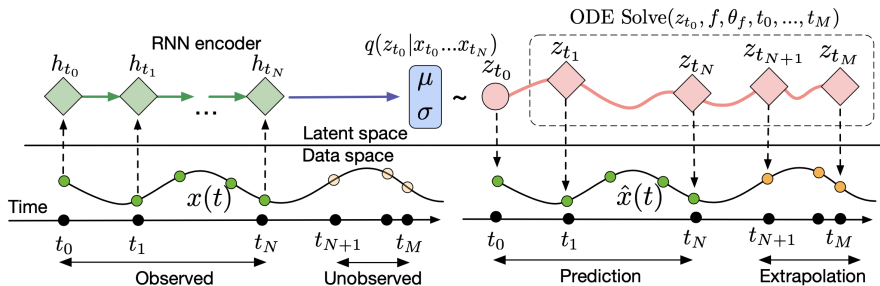
Continuous latent trajectory modeled by NODE

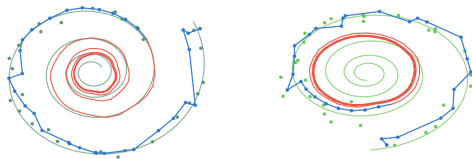
$$\mathbf{z}_{t_0} \sim p(\mathbf{z}_{t_0})$$

$$\mathbf{z}_{t_1}, \mathbf{z}_{t_2}, \dots, \mathbf{z}_{t_N} = \text{ODESolve}(\mathbf{z}_{t_0}, f, \theta_f, t_0, \dots, t_N)$$

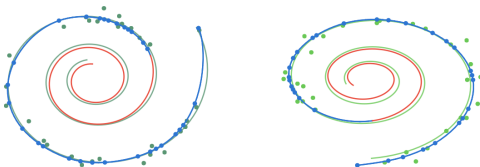
$$\text{each } \mathbf{x}_{t_i} \sim p(\mathbf{x}|\mathbf{z}_{t_i}, \theta_x)$$

$$\text{ELBO} = \sum_{i=1}^M \log p(\mathbf{x}_{t_i}|\mathbf{z}_{t_i}, \theta_x) + \log p(\mathbf{z}_{t_0}) - \log q(\mathbf{z}_{t_0}|\{\mathbf{x}_{t_i}, t_i\}_i, \phi)$$





(a) Recurrent Neural Network



(b) Latent Neural Ordinary Differential Equation



Figure: Fig. 8 and Fig. 9, respectively, in NODE

Continuous change of variables

- Let $z_1 = f(z_0)$ be a bijective transformation and J_f be the Jacobian of f (for scalar z , $J_f = \frac{df}{dz}$). Then we have:

$$p(z_1) = p(z_0) \left| \det J_{f^{-1}} \right| = p(z_0) \left| \frac{1}{\det J_f} \right|$$

$$\log p(z_1) = \log p(z_0) - \log \left| \det J_f \right|$$

- Now let $\frac{dz}{dt} = f(z(t), t)$. Then the change in log probability is an ODE (under some regularity conditions):

$$\frac{\partial \log p(z(t))}{\partial t} = -\text{tr} \left(\frac{df}{dz(t)} \right)$$

Sketch of proof of the Instantaneous version

- Let $z(t + \epsilon) = T_\epsilon(z(t))$, f Lipschitz and $z(t)$ bounded.
- We will need the following results from Taylor expanding T :

$$T \approx z(t) + \epsilon \frac{dz}{dt} = z(t) + \epsilon f(z(t), t)$$

$$\frac{\partial T}{\partial z} = I + \epsilon \frac{\partial f(z(t), t)}{\partial z} + O(\epsilon^2)$$

$$\implies \lim_{\epsilon \rightarrow 0^+} \frac{\partial T}{\partial z} = I, \quad \lim_{\epsilon \rightarrow 0^+} \det \frac{\partial T}{\partial z} = 1$$

$$\lim_{\epsilon \rightarrow 0^+} \frac{\partial}{\partial \epsilon} \frac{\partial T}{\partial z} = \frac{\partial f(z(t), t)}{\partial z}$$

- Full proof in appendix A

Sketch of proof of the Instantaneous version

$$\lim_{\epsilon \rightarrow 0^+} \det \frac{\partial T}{\partial z} = 1, \quad \lim_{\epsilon \rightarrow 0^+} \frac{\partial}{\partial \epsilon} \frac{\partial T}{\partial z} = \frac{\partial f(z(t), t)}{\partial z}, \quad \frac{d}{dt} |A| = |A| \operatorname{tr} \left(A^{-1} \frac{dA}{dt} \right)$$

$$\begin{aligned} \frac{\partial \log p(z(t))}{\partial t} &= \lim_{\epsilon \rightarrow 0^+} \frac{1}{\epsilon} \left[\log p(z(t)) - \log \left| \det \frac{\partial T}{\partial z} \right| - \log p(z(t)) \right] = \\ &= - \lim_{\epsilon \rightarrow 0^+} \frac{1}{\epsilon} \left[\log \left| \det \frac{\partial T}{\partial z} \right| \right] = - \lim_{\epsilon \rightarrow 0^+} \frac{\partial}{\partial \epsilon} \log \left| \det \frac{\partial T}{\partial z} \right| = \\ &= - \lim_{\epsilon \rightarrow 0^+} \frac{\partial}{\partial \epsilon} \left| \det \frac{\partial T}{\partial z} \right| \cdot \lim_{\epsilon \rightarrow 0^+} \left(\det \frac{\partial T}{\partial z} \right)^{-1} = - \lim_{\epsilon \rightarrow 0^+} \frac{\partial}{\partial \epsilon} \left| \det \frac{\partial T}{\partial z} \right| = \\ &= - \lim_{\epsilon \rightarrow 0^+} \left| \det \frac{\partial T}{\partial z} \right| \operatorname{tr} \left(\lim_{\epsilon \rightarrow 0^+} \frac{\partial T^{-1}}{\partial z} \cdot \lim_{\epsilon \rightarrow 0^+} \frac{\partial}{\partial \epsilon} \frac{\partial T}{\partial z} \right) = - \operatorname{tr} \left(\frac{\partial f(z(t), t)}{\partial z} \right) \end{aligned}$$

Normalizing flows⁶

Let f be invertible, and $z' = f(z)$. Then:

$$q(z') = q(z) \left| \det \frac{\partial f^{-1}}{\partial z'} \right| = q(z) \left| \det \frac{\partial f}{\partial z} \right|^{-1}$$

$$z_K = f_K \circ \dots \circ f_2 \circ f_1(z_0)$$

$$\ln q_K(z_K) = \ln q_0(z_0) - \sum_{k=1}^K \ln \det \left| \frac{\partial f_k}{\partial z_k} \right|$$

Example – planar flow:

$$z(t+1) = z(t) + uh(w^T z(t) + b)$$

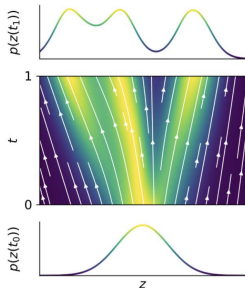
$$\left| \det \frac{dz(t+1)}{dz} \right| = \left| \det \left(I + u \frac{\partial h^T}{\partial z} \right) \right| = \left| 1 + u^T \frac{\partial h}{\partial z} \right|$$

$$\log p(z(t+1)) = \log p(z(t)) - \log \left| 1 + u^T \frac{\partial h}{\partial z} \right|$$

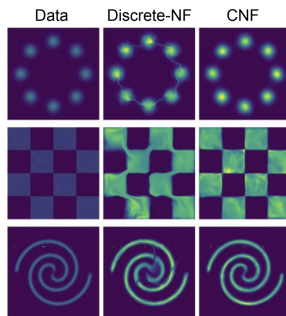
⁶Danilo Jimenez Rezende and Shakir Mohamed. “Variational inference with normalizing flows”. In: *arXiv preprint arXiv:1505.05770* (2015).

Continuous normalizing flows

$$\frac{dz(t)}{dt} = uh(w^T z(t) + b)$$
$$\frac{\partial \log p(z(t))}{\partial t} = -\text{tr}\left(\frac{df}{dz(t)}\right) = -u^T \frac{\partial h}{\partial z(t)}$$
$$\log p(z_T) = \log p(z_0) - \int_0^T \text{tr}\left(\frac{\partial h}{\partial z}\right) dt$$



2D:



Limitations

- ODE becomes more demanding to compute over time
- Can't control duration of training (NFE increase cannot be predicted)
- Dimensionality of input/start and output/end must be the same
- Toy data experiments
- Continuity of vector fields limits possible flows

Table of Contents

1 Introduction

- Deep Learning Revolution
- Residual Networks
- Ordinary Differential Equations

2 Neural ODEs

- Motivation
- Training
- Applications

3 Augmented Neural ODEs

- Motivation
- Approach

NODE can fail for trivial functions

Let g be any function such that $g(-1) = 1$ and $g(1) = -1$. Then NODE cannot represent it.

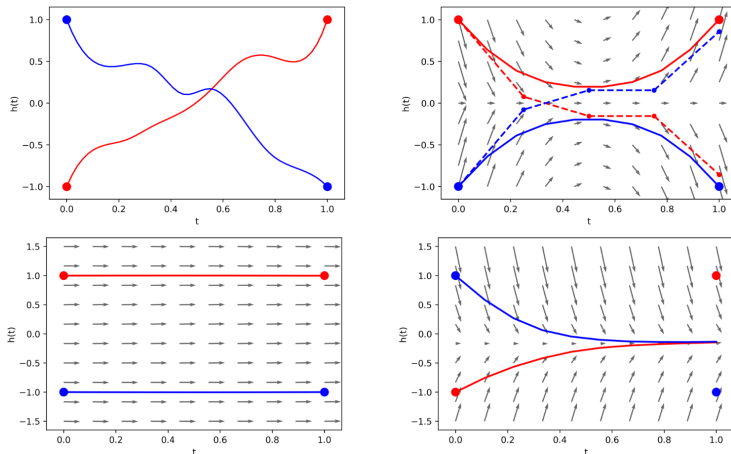


Figure: Fig. 3 in ANODE

Limitations of NODE

- Why is this the case?
- The trajectories intersect. The initial value problem means existing solutions are unique (under regularity conditions)
- Interestingly, ResNets and the Euler method can solve it. Why?
- Discretization errors allow for jumps.

NODE general failure class of functions

- Let's stack a linear layer on the output
- Let $0 < r_1 < r_2 < r_3$, and $g(x)$ such that:

$$\begin{cases} g(\mathbf{x}) = -1 & \text{if } \|\mathbf{x}\| \leq r_1 \\ g(\mathbf{x}) = 1 & \text{if } r_2 \leq \|\mathbf{x}\| \leq r_3 \end{cases}$$

- Then for any dimension d , NODE cannot represent $g(x)$. Why?
- NODE is a homeomorphism (preserves topology)

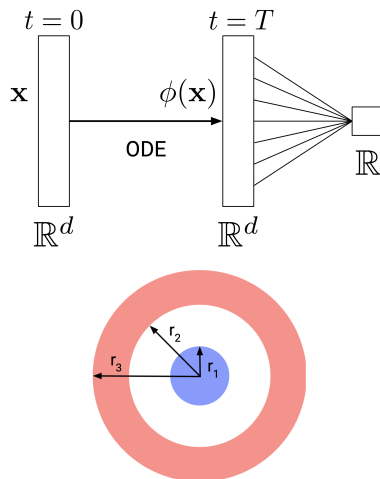
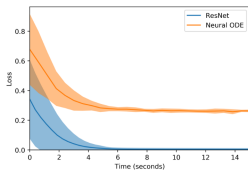
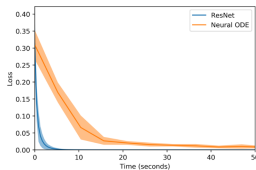


Figure: Fig. 2 and 4 in ANODE

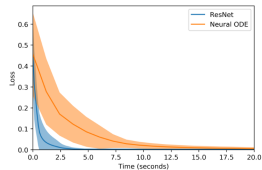
NODE in practice



(a) $g(\mathbf{x})$ in $d=1$



(b) $g(\mathbf{x})$ in $d=2$



(c) Separable function in $d=2$

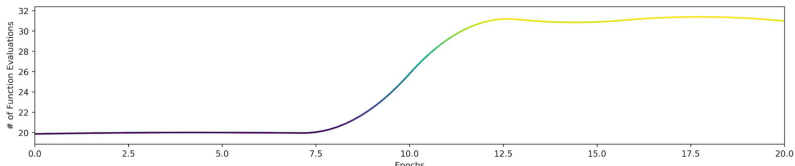
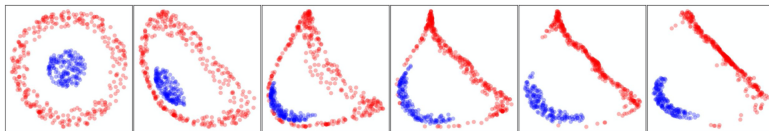


Figure: Fig. 5 and 6 in ANODE

Augmented neural ODEs⁷

- Should we discard NODEs? No
- Can we avoid intersections and achieve linear separability? Yes
- How? Augment the space with extra dimensions
- Concatenate $a \in \mathbb{R}^p$ to $h \in \mathbb{R}^d$ with initial condition $a(0) = 0$ and solve the $\mathbb{R}^{d+p}$ ODE:

$$\frac{d}{dt} \begin{bmatrix} \mathbf{h}(t) \\ \mathbf{a}(t) \end{bmatrix} = \mathbf{f} \left(\begin{bmatrix} \mathbf{h}(t) \\ \mathbf{a}(t) \end{bmatrix}, t \right), \quad \begin{bmatrix} \mathbf{h}(0) \\ \mathbf{a}(0) \end{bmatrix} = \begin{bmatrix} \mathbf{x} \\ \mathbf{0} \end{bmatrix}$$

⁷Emilien Dupont, Arnaud Doucet, and Yee Whye Teh. “Augmented neural odes”. In: *arXiv preprint arXiv:1904.01681* (2019).

Comparing flows

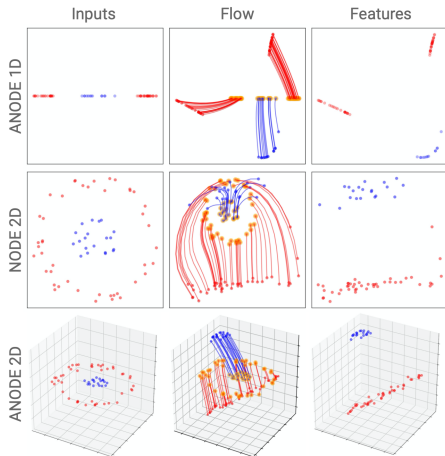
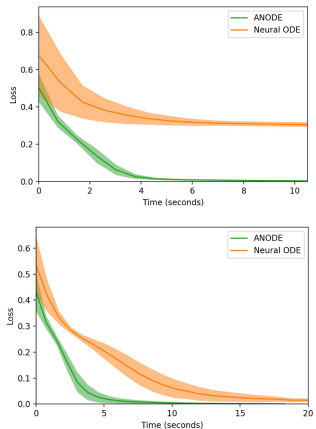


Figure: Fig. 7 in ANODE

Flow complexity and NFE over time

- Extra dimensions allow for simpler flows, which means a smaller number of function evaluations (NFEs)

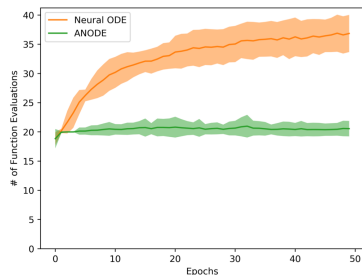
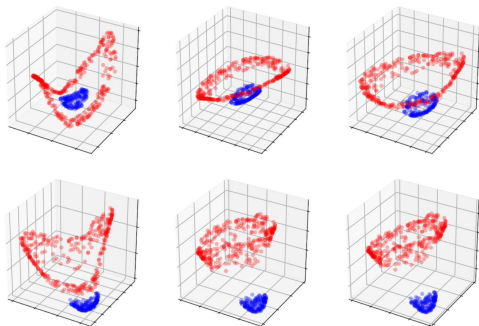


Figure: Fig. 8 in ANODE

ImageNet and stability

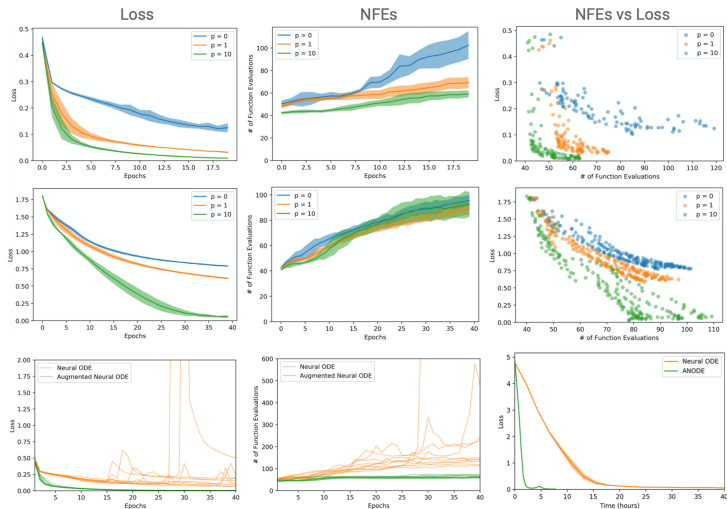


Figure: Fig. 10 and 12 in ANODE. Top row is MNIST, middle is CIFAR.

- Neural ODE provide a new way to adaptively train "deep", "equal width" ResNets
- Training allows setting error tolerances
- Can be made powerful by augmenting the dimensionality
- However, even ANODE cannot model uncertainty
- Continuous flow and other applications in the next presentation

References

- Chen, Tian Qi et al. “Neural ordinary differential equations”. In: *Advances in neural information processing systems*. 2018, pp. 6571–6583.
- Dupont, Emilien, Arnaud Doucet, and Yee Whye Teh. “Augmented neural odes”. In: *arXiv preprint arXiv:1904.01681* (2019).
- He, Kaiming et al. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural networks 2.5* (1989), pp. 359–366.
- Lu, Zhou et al. “The expressive power of neural networks: A view from the width”. In: *Advances in neural information processing systems*. 2017, pp. 6231–6239.
- Rezende, Danilo Jimenez and Shakir Mohamed. “Variational inference with normalizing flows”. In: *arXiv preprint arXiv:1505.05770* (2015).

Thank You!

Questions?